

AIDED

Assisted Analysis of the Dynamic Electron Density

J. Robert Michael, PhD

June 21, 2025

Contents

1	Introduction	1
2	Static electron density	1
2.1	Basis expansion	2
3	Mean-square displacement amplitudes (MSDA)	2
4	Dynamic electron density	3
4.1	Primitive s - s unit	3
4.2	Primitive s - p unit	3
4.3	General Cartesian exponents	3
4.4	Reference NumPy implementation	4

1 Introduction

AIDED evaluates both the static and the vibration-averaged (“dynamic”) electron density of molecular systems, together with derived topological properties such as critical points and zero-flux surfaces. A key design goal is to *reuse the primitive-Gaussian basis* exported by standard quantum-chemistry programs so that no extra integrals are needed.

2 Static electron density

The (static) electron density $\rho(\mathbf{r})$ of a system with N electrons is¹

$$\rho(\mathbf{r}) = N \int \Psi(\mathbf{r}, \mathbf{r}_2, \dots, \mathbf{r}_N; \mathbf{R}) \Psi^*(\mathbf{r}, \mathbf{r}_2, \dots, \mathbf{r}_N; \mathbf{R}) d\mathbf{r}_2 \cdots d\mathbf{r}_N,$$

where Ψ is the ground-state wave-function at the fixed nuclear geometry $\mathbf{R}$.

¹Spin is omitted for clarity.

2.1 Basis expansion

Each *atomic* orbital (AO) is a contraction of primitive Gaussians g_k :

$$\phi_j(\mathbf{r}) = \sum_k g_{jk} g_k(\mathbf{r}). \quad (1)$$

A molecular orbital (MO) is an AO-linear combination,

$$\Psi_i(\mathbf{r}) = \sum_\mu c_{i\mu} \phi_\mu(\mathbf{r}) = \sum_k C_{ik} g_k(\mathbf{r}), \quad C_{ik} = \sum_\mu c_{i\mu} g_{\mu k}. \quad (2)$$

The AO-resolved one-particle density matrix is

$$P_{\mu\nu} = 2 \sum_i n_i c_{i\mu} c_{i\nu},$$

with $n_i \in \{0, 1, 2\}$ the MO occupations. Inserting (1) gives the primitive-basis density matrix

$$D_{kl} = \sum_{\mu,\nu} g_{\mu k} P_{\mu\nu} g_{\nu l} = \sum_i n_i C_{ik} C_{il}, \quad (3)$$

so that the density at any grid point is the simple quadratic form

$$\rho(\mathbf{r}) = \sum_{k,l} D_{kl} g_k(\mathbf{r}) g_l(\mathbf{r}) = \mathbf{g}^\top(\mathbf{r}) D \mathbf{g}(\mathbf{r}).$$

The AIDED code builds D once from the `*.wfn` MO coefficients, then evaluates $\rho(\mathbf{r})$ on the desired grid.

3 Mean-square displacement amplitudes (MSDA)

Normal-mode analysis of a molecule with M nuclei gives a $(3M)$ -variate Gaussian probability distribution for nuclear displacements $\mathbf{u} = \mathbf{R} - \mathbf{R}^0$:

$$P(\mathbf{u}) = \frac{\exp(-\frac{1}{2} \mathbf{u}^\top \mathbf{U}^{-1} \mathbf{u})}{(2\pi)^{3M/2} \sqrt{\det \mathbf{U}}}, \quad (4)$$

where $\mathbf{U} = \langle \mathbf{u} \mathbf{u}^\top \rangle_T$ is the Cartesian MSDA matrix. Diagonalising the mass-weighted Hessian yields

$$\mathbf{U} = \mathbf{L} \mathbf{\Delta} \mathbf{L}^\top, \quad \Delta_{jj} = \frac{h}{8\pi^2 \nu_j} \coth\left(\frac{h\nu_j}{2k_B T}\right),$$

implemented verbatim in `gaussian.py::gen_msda`.

4 Dynamic electron density

4.1 Primitive s - s unit

$$\gamma = \alpha + \beta, \quad \mathbf{c} = \frac{\alpha \mathbf{a} + \beta \mathbf{b}}{\gamma}, \quad \mathbf{G} = \gamma I_3, \quad \mathbf{W} = (\mathbf{G}^{-1} + \mathbf{U})^{-1}.$$

$$\boxed{g_{ss}^{\text{dyn}}(\mathbf{x}) = N_{\alpha\beta} \exp\left[-\frac{1}{2}(\mathbf{x} - \mathbf{c})^T \mathbf{W}^{-1}(\mathbf{x} - \mathbf{c}) - \frac{1}{2} \frac{\alpha\beta}{\gamma} |\mathbf{a} - \mathbf{b}|^2\right]}, \quad N_{\alpha\beta} = [\gamma^{-3} \det \mathbf{W}]^{1/2}. \quad (5)$$

4.2 Primitive s - p unit

The Cartesian p factor is generated by differentiating g_{ss}^{dyn} with respect to one nuclear centre (choose $\mathbf{b}$ without loss of generality). Because

$$\partial_{b_\nu} \mathbf{c} = \frac{\beta}{\gamma} \mathbf{e}_\nu, \quad \partial_{b_\nu} |\mathbf{a} - \mathbf{b}|^2 = -2(\mathbf{a}_\nu - \mathbf{b}_\nu),$$

one obtains

$$g_{sp_\nu}^{\text{dyn}}(\mathbf{x}) \equiv -\frac{1}{2\beta} \partial_{b_\nu} g_{ss}^{\text{dyn}}(\mathbf{x}) \quad (6)$$

$$= \underbrace{\left[\frac{\beta}{\gamma} (\mathbf{W}^{-1}(\mathbf{x} - \mathbf{c}))_\nu + \frac{\alpha\beta}{\gamma} (\mathbf{a}_\nu - \mathbf{b}_\nu) \right]}_{L_\nu(\mathbf{x})} g_{ss}^{\text{dyn}}(\mathbf{x}). \quad (7)$$

Linear prefactor. The bracketed quantity

$$L_\nu(\mathbf{x}) = \frac{\beta}{\gamma} (\mathbf{W}^{-1}(\mathbf{x} - \mathbf{c}))_\nu + \frac{\alpha\beta}{\gamma} (\mathbf{a}_\nu - \mathbf{b}_\nu)$$

is exactly the “ C_4, C_5, C_6 ” coefficient that the FORTRAN and NumPy kernels store for p_x, p_y, p_z on centre j .

(Interchanging $\alpha \leftrightarrow \beta$ and $\mathbf{a} \leftrightarrow \mathbf{b}$ gives the p factor on centre i).

4.3 General Cartesian exponents

For arbitrary Cartesian angular momenta $\mathbf{L}_i = (\ell_i, m_i, n_i)$ and $\mathbf{L}_j = (\ell_j, m_j, n_j)$, repeat the differentiation:

$$g_{\mathbf{L}_i; \mathbf{L}_j}^{\text{dyn}}(\mathbf{x}) = \prod_{\mu=1}^3 (\partial_{a_\mu})_i^{\ell_i^{(\mu)}} \prod_{\nu=1}^3 (\partial_{b_\nu})_j^{\ell_j^{(\nu)}} g_{ss}^{\text{dyn}}(\mathbf{x}).$$

All such derivatives factorise into

$$g_{\mathbf{L}_i; \mathbf{L}_j}^{\text{dyn}}(\mathbf{x}) = g_{ss}^{\text{dyn}}(\mathbf{x}) \prod_{\mu=1}^3 [C_\mu(\mathbf{x})]_i^{\ell_i^{(\mu)}} \prod_{\nu=1}^3 [C_{3+\nu}(\mathbf{x})]_j^{\ell_j^{(\nu)}},$$

where the six *linear* terms are

$$C_{1,2,3}(\mathbf{x}) = \frac{\alpha}{\gamma} \mathbf{W}^{-1}(\mathbf{x}-\mathbf{c}) - \frac{\alpha\beta}{\gamma} (\mathbf{a}-\mathbf{b}), \quad C_{4,5,6}(\mathbf{x}) = \frac{\beta}{\gamma} \mathbf{W}^{-1}(\mathbf{x}-\mathbf{c}) + \frac{\alpha\beta}{\gamma} (\mathbf{a}-\mathbf{b}),$$

i.e. the p -type prefactors on centres i and j . Higher- l Gaussians follow by simply raising these linear factors to the required powers—the same rule implemented in ‘gen_gcon’ and in the NumPy kernel of Listing 1.

4.4 Reference NumPy implementation

Listing 1: Vectorised NumPy kernel matching Eq. (10)

```

1 import numpy as np
2
3 def dynamic_gaussian(x, A, B, alpha, beta, U,
4                      lmn_i=(0,0,0), lmn_j=(0,0,0)):
5     gamma = alpha + beta
6     C      = (alpha*A + beta*B) / gamma
7     W      = np.linalg.inv(np.eye(3)/gamma + U)
8
9     prefactor = (gamma**-3 * np.linalg.det(W))**0.5
10    Eg  = np.exp(-0.5*alpha*beta/gamma * np.dot(A-B, A-B))
11    expo = np.exp(-0.5 * (x-C) @ np.linalg.inv(W) @ (x-C))
12    g_ss = prefactor * Eg * expo
13
14    # Ci factors omitted for brevity ...
15    return g_ss

```

NOTES

This document is based off of my PhD thesis [3] and is based off of Scheringer [1] which is referenced in [2]. and is originally implemented in the DENPROP package [4].

This document was generated with the help of AI LLM models with guidance from my previous work.

References

- [1] J. Scheringer and H. Reitz, *Direct Calculation of Dynamic Densities*, *Acta Crystallographica A*, **32** (1976) 271–276.
- [2] J. R. Michael and T. Koritsanszky, Validation of convolution approximation to the thermal-average electron density, *Journal of Mathematical Chemistry* **53** (2015) 250–259.

- [3] J. R. Michael, *Direct Calculation of Dynamic Electron Density: Theoretical Development and Experimental Validation*, Ph.D. thesis, Middle Tennessee State University (2013).
- [4] A. Volkov, T. Koritsanszky, M. Chodkiewicz, and H. F. King On the basis-set dependence of local and integrated electron-density properties: Application of a new computer program for quantum-chemical density analysis, *Journal of Computational Chemistry* **25** (2004) 166–183. doi:10.1002/jcc.10434.